

Advanced Data Management

Ontologies and Knowledge Engineering

Domenico Fabio Savo

*Corso di laurea magistrale
INGEGNERIA INFORMATICA
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione
University of Bergamo*

Outline of the course

1. Introduction to propositional logic
2. Introduction to First-order Logic
3. Relational Calculus
4. Information Integration Systems (IIS)
5. Logical formalization of IISs
6. Mapping between **Global Schema** e **Data Sources**
7. Incomplete information databases
8. Query answering over IISs
9. **Ontology-Based Data Management (OBDM)**
10. Description Logic Ontologies
11. Query answering in ontologies
12. Query answering in OBDM
13. **Knowledge Engineering**

Introduction to Ontologies

What is an Ontology?

Definition: An **ontology** is a formal, explicit specification of a shared conceptualization.

Key aspects:

- **Formal:** machine-readable
- **Explicit:** concepts, properties, relations, functions, constraints, and axioms are explicitly defined
- **Shared:** consensual knowledge
- **Conceptualization:** abstract model of phenomena in the world

Origins in philosophy, adapted for computer science and AI

Ontologies in AI and Knowledge Representation

Ontologies help in:

- **Knowledge Sharing:** Enable communication between systems.
- **Knowledge Reuse:** Reusable in different contexts.
- **Automated Reasoning:** Using logical rules to infer new knowledge.

Example: Reuse of ontologies in e-commerce and healthcare.

Why Ontologies?

- Enable **knowledge sharing**
 - Provide a **common understanding** of a domain
 - Make domain assumptions **explicit**
 - Separate **domain knowledge** from operational knowledge
 - Enable **reuse** of domain knowledge
 - Facilitate **interoperability** between systems
 - Automated **Reasoning**, using logical rules to infer new knowledge.
-

Components of Ontologies

- **Concepts**: represent sets of objects
 - **Roles**: ways in which classes and instances can be related to one another
 - **Instances**: specific membership of objects in classes and relationship
 - **Attributes**: relationships between concepts and concrete domains
 - **Axioms**: assertions in a logical form about the classes, instances, and relations
-

Components Example

Domain: University

- **Classes:** Person, Student, Professor, Course
 - **Roles:** teaches, enrolledIn
 - **Instances:** Student(John), Course(CS), enrolledIn (John, CS)
 - **Properties:** name, courseCode
 - **Axioms:** Professor $\sqsubseteq$ Person, Student $\sqsubseteq$ $\exists$ enrolledIn
-

Types of Ontologies

- **Upper (Top-level) Ontologies**: describe very general concepts applicable across domains. Examples: SUMO, BFO, DOLCE
 - **Domain Ontologies**: represent concepts in a specific domain (e.g., medicine, finance)
 - **Task Ontologies**: describe vocabulary for a specific task or activity (e.g., ontologies for web search)
 - **Application Ontologies**: combine domain and task ontologies for a specific application
-

Ontology Languages

Mathematical (logical) Languages:

- **FOL**
- **Description Logics**
- **Datalog**
- ...

Concrete Languages (machine readable):

- **RDF (Resource Description Framework)**: subject-predicate-object triples
- **RDFS (RDF Schema)**: extends RDF with basic ontological modeling.
- **OWL 2 (Web Ontology Language)**: based on Description Logics, equipped with profiles (OWL-Lite, OWL-DL, OWL-Full), **W3C standard**.

Example of a Simple Ontology in DL

TBox:

$\delta(\text{name}) \sqsubseteq \text{Person}$
 $\rho(\text{name}) \sqsubseteq \text{xsd:string}$
 $\text{Professor} \sqsubseteq \text{Person}$
 $\text{Student} \sqsubseteq \text{Person}$
 $\text{Student} \sqsubseteq \exists \text{enrolledIn}$

ABox:

$\text{Course}(\text{CS})$
 $\text{Student}(\text{john})$
 $\text{enrolledIn}(\text{john}, \text{CS})$

Example of a Simple Ontology in OWL

```
Declaration(Class(Course))
Declaration(Class(Person))
Declaration(Class(Professor))
Declaration(Class(Student))
Declaration(ObjectProperty(enrolledIn))
Declaration(ObjectProperty(teaches))
Declaration(DataProperty(name))
Declaration(NamedIndividual(CS))
Declaration(NamedIndividual(john))

DataPropertyDomain(name Person)
DataPropertyRange(name xsd:string)

SubClassOf(Professor Person)
SubClassOf(Student Person)
SubClassOf(Student ObjectSomeValuesFrom(enrolledIn owl:Thing))

ClassAssertion(Course CS)

ClassAssertion(Student john)
ObjectPropertyAssertion(enrolledIn john CS)
```

Knowledge Engineering

What is Knowledge Engineering?

Definition: Process of designing, building, and managing knowledge-based systems.

Key Activities:

- Knowledge acquisition
 - Knowledge modeling
 - Knowledge validation and maintenance
-

Knowledge Engineering Cycle

Key steps:

1. **Domain analysis and Knowledge Acquisition**: gathering knowledge from experts and literature.
2. **Conceptualization**: structuring knowledge into models.
3. **Ontology development**: using formal languages like OWL or RDF.
4. **Ontology population**: specifying instances of concepts, roles and attributes
5. **Ontology evaluation and refinement**: testing the accuracy and efficiency of the ontology.
6. **Ontology maintenance and evolution**: Keeping ontologies up to date.

1 - Domain Analysis and Knowledge Acquisition

The first step requires to identify the domain and scope of the ontology

Gather domain knowledge from:

- Domain experts
- Existing ontologies
- Textbooks and scientific literature
- Databases and other structured sources

Techniques:

- Interviews with domain experts
 - Document analysis
 - Collaborative workshops
-

2 - Conceptualization

- Organize and structure the acquired knowledge
 - Identify key **concepts**, **relations**, and **properties**
 - Create **informal** representations:
 - Concept maps
 - UML diagrams
 - Glossaries and taxonomies
 - **Validate** conceptualization with domain experts
-

3 - Ontology Design and Development

- Choose an appropriate **ontology language** (e.g., OWL)
 - Formalize the conceptualization:
 - Define **classes** and class **hierarchies**
 - Define **properties** and **relations**
 - Create **instances**
 - Specify **axioms** and rules
 - Use ontology development tools (e.g., **Protégé**, Eddy, etc.)
-

4 - Ontology Population

- Add **instances** (individuals) to the ontology
 - Methods:
 - Manual population by domain experts
 - Semi-automatic population using:
 - Natural Language Processing techniques
 - Machine Learning algorithms
 - Automatic population from structured data sources (e.g. OBDM)
 - Ensure **consistency** and **quality** of added instances
-

5 - Ontology Evaluation and Refinement

- Verify the ontology meets its **requirements** and competency questions
 - **Evaluation criteria:**
 - **Consistency**
 - **Completeness**
 - **Conciseness**
 - **Expandability**
 - **Techniques:**
 - Automated reasoning
 - Expert review
 - Application-based evaluation
-

6 - Ontology Maintenance and Evolution

Ontologies are dynamic and need to evolve

- Reasons for evolution:
 - **Changes in the domain**
 - **New requirements**
 - **Errors or inconsistencies discovered**
 - Versioning and change management
 - Collaborative ontology development and maintenance
-

Approaches for Building Ontologies

- **Manual Engineering:** ontologies developed by experts.
 - **Semi-automatic:** tools assist in suggesting classes/relations.
 - **Fully Automatic:** algorithms extract knowledge (not common due to complexity).
-

Popular Ontology Development Methodologies

Over the years, various methodologies for developing structured ontologies have been proposed.

Some of these are:

- **Methontology**: A structured method including stages of specification, conceptualization, integration, and evaluation.
 - **Ontology Development 101**: Simpler approach focusing on identifying classes, hierarchy, and properties.
 - **NeON Methodology**: Focuses on reuse and networking of ontologies (integration of existing ontologies).
-

Practical Applications of Ontologies

- Semantic Web and Linked Data
 - Knowledge Management Systems
 - Information Retrieval and Question Answering
 - Natural Language Processing
 - Data Integration and Interoperability
 - Decision Support Systems
 - Bioinformatics and Healthcare
 - E-commerce and Recommender Systems
-

Applications and Case Studies

Case Study 1: Biomedical Ontologies

- Ontologies like **SNOMED CT**, **Gene Ontology**, and **HPO** use DL for formalizing medical knowledge.
- **Example**: Defining hierarchical relationships between diseases and symptoms.

Case Study 2: Ontologies in the Semantic Web

- **FOAF (Friend of a Friend)**: An ontology to describe people and their social networks.
 - **Example**: Using DL to infer indirect social relationships.
-

Challenges in Ontology Engineering

- **Scalability:** Large ontologies can become difficult to manage and reason over.
 - **Integration:** Merging ontologies from different domains can lead to conflicts.
 - **Human Expertise:** Requires significant domain knowledge to develop useful ontologies.
-